JWT Structure

A JWT consists of three parts separated by dots (.)

HEADER	PAYLOAD	SIGNATURE
Algorithm & Token Type `{"alg": "HS256", "typ": "JWT"}`	**Claims (User Data)** `{"sub": "user@email.com", "role": "ADMIN", "exp": 1735689600}`	**Verification Hash** `HMACSHA256(base64(header) + '.' + base64(payload), secret)`

Complete JWT Token:
`eyJhbGciOiJIUzI1NiIsInR5cCIkIkpXVCJ9.eyJzdWIiOiJlcnQgQGVtYWlsLmNvbSIsInJvbGUiOiJNOW5jbmR.5fiXnwE3...`

How JWT Works: Authentication Flow

1. **User Login:** Client sends credentials (username/password) to server →
2. **Verify Credentials:** Server validates credentials against database →
3. **Generate JWT:** Server creates JWT with user data and signs it with secret key →
4. **Return Token:** Server sends JWT back to client →
5. **Store Token:** Client stores JWT (localStorage/sessionStorage/cookie) →
6. **Send with Requests:** Client includes JWT in Authorization header for API calls →
7. **Verify Token:** Server validates JWT signature and extracts user data →
8. **Grant Access:** If valid, server processes request and returns response

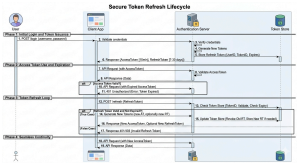
Rule	Standard
Line length	88 characters (Black formatter default)
Indentation	4 spaces (no tabs)
Imports	Grouped: stdlib, third-party, local. Alphabetized within groups.
Quotes	Double quotes for strings (Black formatter default)
Trailing commas	Always on multi-line collections and function signatures
Blank lines	2 between top-level definitions, 1 between methods
Formatter	Black (enforced in CI)
Linter	Ruff (replaces flake8, isort, pyupgrade)

Purpose of JWT Refresh Token

A Refresh Token is a special token used to obtain new access tokens without requiring the user to re-authenticate. It solves a critical security vs. usability trade-off in JWT-based authentication.

The Problem JWT Refresh Tokens Solve
Access Tokens Alone Have Limitations
If you only use access tokens:

- Short-lived tokens (5-15 minutes) = good security, bad UX (users constantly re-login)
- Long-lived tokens (days/weeks) = bad security (if stolen, attacker has long-term access)



Recommended Approach: Split Storage
The best practice is to split tokens based on their purpose:

Access Token → Memory (most secure)
Refresh Token → HttpOnly Cookie (secure + persistent)

Application State - Redux, Vuex, or a simple variable

An HttpOnly cookie is a security flag added to an HTTP response header that prevents client-side scripts (like JavaScript) from accessing the cookie via document.cookie. It is primarily used to protect sensitive data, such as session tokens, from theft via Cross-Site Scripting (XSS) attacks.

1. Where is the Token Store?

On the Client (The Browser/Mobile App)

The client needs to store tokens so it can stay logged in.

- **Access Token:** Usually kept in **Memory** (a JavaScript variable). Since it's short-lived, it's okay if it disappears when the tab closes.
- **Refresh Token:** Needs to persist.
  - **Best Practice (Web):** Store it in an **HttpOnly, Secure Cookie**. This prevents JavaScript from reading it, which stops XSS (Cross-Site Scripting) attacks from stealing your session.
  - **Mobile:** Use **EncryptedSharedPreferences** (Android) or **Keychain** (iOS).

On the Server (The Backend)

The server needs a "source of truth" to know if a Refresh Token is still valid.

- **Database (PostgreSQL/MySQL):** A table like `refresh_tokens` stores the token ID, who it belongs to, and its expiration date.
- **Cache (Redis):** This is the most popular choice because it is incredibly fast and has built-in **TTL (Time-To-Live)**. You can set the token to auto-delete the moment it expires.

2. How do we revoke the old Refresh Token?

Revocation is critical for security—for example, when a user logs out or if they click "Sign out of all devices."

Method A: Manual Revocation (The "Logout")

When a user clicks logout, the Client sends a request to a `/revoke` or `/logout` endpoint.

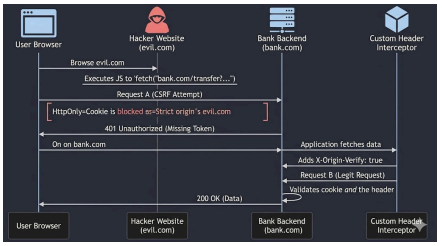
1. **Client** sends the Refresh Token to the Server.
2. **Server** looks up that token in the Database/Redis.
3. **Server** deletes the record (or marks it as `is_revoked = true`).
4. **Client** clears the cookie or local storage.

Method B: Refresh Token Rotation (Automatic)

This is the "gold standard" for security. Every time the client uses a Refresh Token to get a new Access Token, the server **kills the old Refresh Token** and issues a **brand new one**.

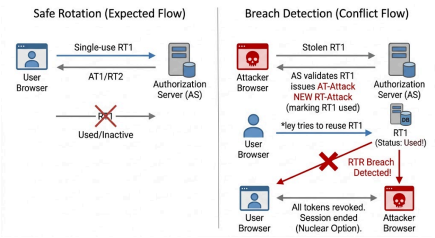
> **The "Detection" Trick:** If an attacker steals a Refresh Token and uses it, but then the *legitimate* user also tries to use that same (now old) token, the server sees the "reuse." Because the server knows that token should have been rotated, it immediately revokes **all** tokens for that user, forcing a fresh login.

https://www.insecure.in/blog/xss-vs-csrf



XSS VS CSRF

Feature	Cross-Site Scripting (XSS)	Cross-Site Request Forgery (CSRF)
User Interaction	Does not require user interaction beyond visiting a compromised site.	Typically involves the user performing an action.
Session Requirement	Does not require an active session with the targeted site.	Requires the user to have an active session with the targeted site.
Trust Exploitation	Exploits the trust a user has in a website.	Exploits the trust a website has in a user's browser.
Scope of Actions	Can execute arbitrary scripts.	Restricted to actions that a user can perform.
Attack Initiation	Requires a vulnerability in the website.	Requires the user to access a malicious page or click a link.
Data Access	Can send and receive HTTP requests and responses.	Can only send HTTP requests and cannot view the response.


Safe Rotation (Expected Flow)
Breach Detection (Conflict Flow)
Single-use RT1
User Browser
AT1/RT2
Authorization Server (AS)
RT1
Used/Inactive
Stolen RT1
Attacker Browser
AS validates RT1 issues AT-Attack NEW RT-Attack (marking RT1 used)
Authorization Server (AS)
*ley tries to reuse RT1
User Browser
RT1 (Status: Used!)
RTR Breach Detected!
User Browser
All tokens revoked. Session ended (Nuclear Option).
Attacker Browser